



The National Teachers College
629 J. Nepomuceno St., Quiapo, Manila

ITELECTIVE 1 for Sustainable Development Goal

ITELECTIVE 1 for Sustainable Development Goal

Submitted by:

Aguirre, Sean Xander C.

Iloseo, Allen Gene U.

Vera, Meljohn

Instructor:

Justine Neypes

Date Submitted

May 22, 2026

Project Overview and Chosen UN SDG Target

Application developed to support the objectives of United Nations Sustainable Development Goal 11 (Sustainable Cities and Communities). The system focuses on improving the management and monitoring of building inspection records through a centralized and organized database application.

The application is designed to let users view information related to building records and inspection results, while allowing administrators to modify, update, and manage the data. Through the use of a computerized system, inspection data can be stored, retrieved, monitored, and maintained more efficiently compared to manual record-keeping methods.

The system uses Microsoft SQL Server as its database and applies ADO.NET for database connectivity and data management. It also includes essential system features such as role-based login access, CRUD operations, report generation, input validation, and error handling to ensure data reliability and usability.

This project is aligned with SDG 11 because safe and properly monitored buildings contribute to the development of safer, more resilient, and sustainable communities. By organizing building inspection records into a single management system, the application can assist in identifying unsafe structures, monitoring inspection compliance, and supporting better decision-making for urban safety and development.

The project also serves as an application of the programming, database management, and software development concepts discussed throughout the course. The system aims to provide a practical and manageable solution for organizing building inspection information while demonstrating the integration of VB.NET, SQL Server, and Windows Forms Application Dev.

Functional Requirements and Non-Functional Requirements

These requirements serve as a guide in the development of the “Safe City: Building Safety Inspection Management System” to ensure that the application meets user needs and supports efficient management of building inspection information.

Functional Requirements

FR1. The system must provide a secure login form with role-based access for Administrator and User accounts.

FR2. The system must allow users to perform Create, Read, Update, and Delete (CRUD) operations for building records, owner information, and inspection records.

FR3. The system must store and retrieve data using Microsoft SQL Server through ADO.NET database connectivity.

FR4. The system must display organized records using DataGridView controls for easier viewing and management of information.

FR5. The system must allow administrators to manage user accounts and system access permissions.

FR6. The system must generate inspection reports, including summaries of passed and failed building inspections.

FR7. The system must implement input validation to prevent incomplete, incorrect, or invalid data entries.

FR8. The system must provide search and filtering functions for faster access to records and inspection information.

FR9. The system must maintain proper relationships between database tables such as Users, Owners, Buildings, and Inspections.

FR10. The system must display confirmation prompts and error messages using MessageBox for better user interaction and system feedback.

Non-Functional Requirements

NFR1. The system must maintain a clean, organized, and user-friendly interface with consistent layouts, readable fonts, and proper control naming conventions.

NFR2. The application must provide reliable database connectivity and stable performance during CRUD operations and report generation.

NFR3. The system must implement proper exception handling using Try-Catch-Finally statements to prevent unexpected errors and application crashes.

NFR4. The application must maintain an organized and modularized code structure to improve maintainability and readability.

NFR5. The system must support efficient data retrieval and quick loading of records from the SQL Server database.

NFR6. The application must apply role-based access restrictions to ensure basic system security and controlled access to features.

NFR7. The system must ensure data accuracy by applying required-field validation, numeric-only validation, and proper database constraints.

NFR8. The application must generate readable and professional reports suitable for viewing, printing, and documentation purposes.

Entity Relationship Diagram (ERD)

BuildingSafety_DB		
building_id 🔗		int
building_name	varchar(100)	NN
address	varchar(200)	NN
floors		int
safety_score		int
safety_status		varchar(50)

The table consists of a Single Entity [BuildingSafety_DB]. Due to the simplicity of the database, the table will act as a snapshot of the record of data on the specific event. Each row provided in the table acts independently and contains information in the entire building report.

Each row stands on its own, so changing one record won't affect the others. Every row also includes all the key details for a building report, such as the building ID, address, number of floors, safety score, and safety status.

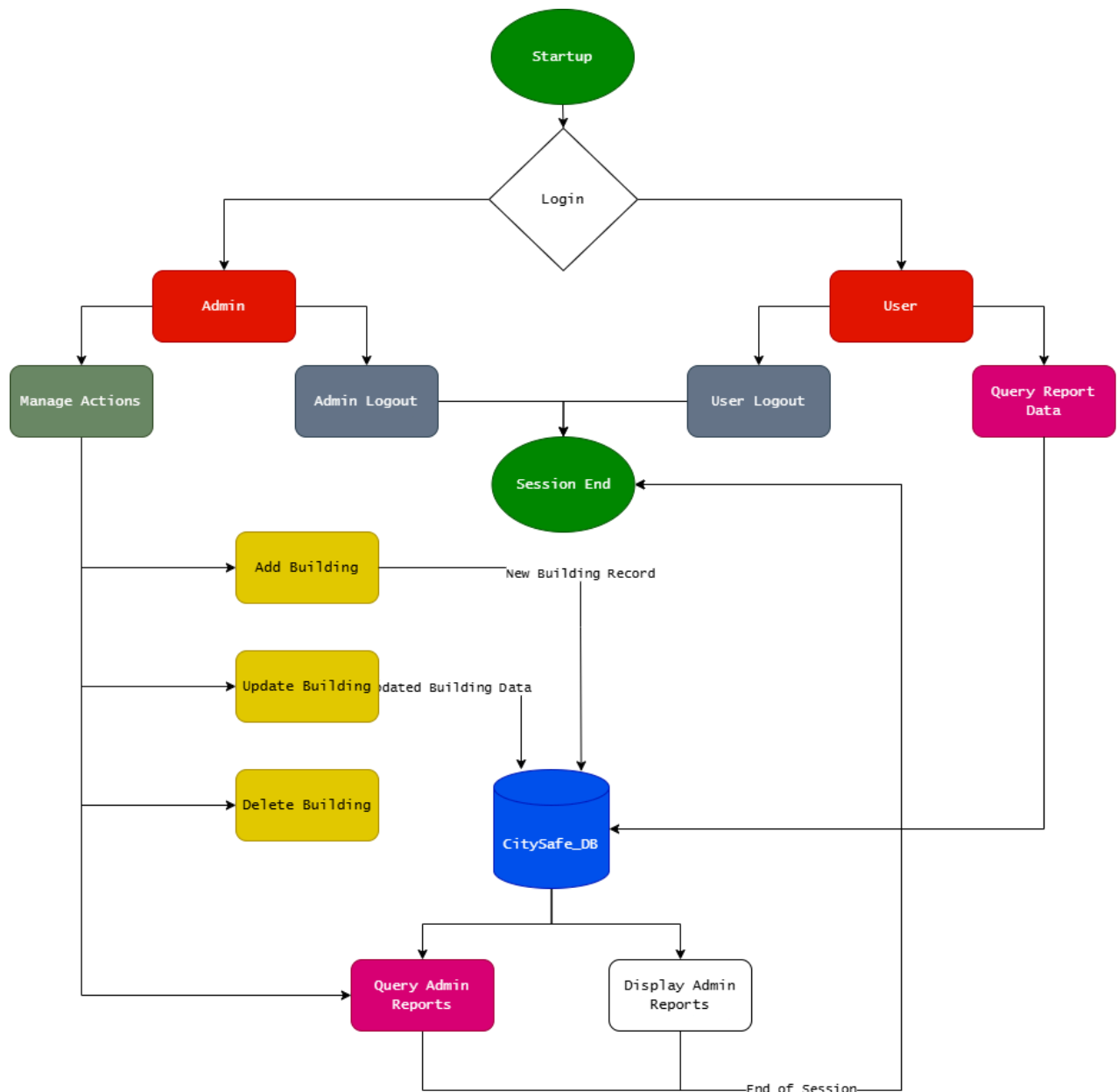
This single-entity snapshot design emphasizes simplicity and ease of use, making it well suited for basic reporting, spreadsheet export, and situations where tracking building details across multiple report cycles is unnecessary.

Data Dictionary: BuildingSafety_DB

Column Name	Data Type	Nullable	Key	Description	Constraint
building_id	INT	NOT NULL	Primary	The system creates a special number for each safety report that is entered.	Must be unique. Must not be null. Auto-increments .
building_name	VARCHAR(100))	NOT NULL	—	The label used for a building in an official way to distinguish it from others.	Must not be null. Must not be blank or empty string.
address	VARCHAR(200))	NOT NULL	—	This refers to the helps identify the location of where the building is located.	Must be a positive integer (greater than 0) if provided.
floors	INT	NULL	—	Total count of story levels contained within the building structure.	Must be between 0 and 100 inclusive if provided.

safety_score	INT	NULL	—	Numerical value indicating the building's overall safety status.	Must be between 0 and 100 inclusive if provided.
safety_status	VARCHAR(50)	NULL	—	A descriptive category that shows the building's safety level according to its associated safety score.	Must be one of: 'SAFE', 'WARNING', 'CRITICAL' if provided

System Architecture



The Safe City System follows a clear sequence that starts at the Startup stage. When the application opens, the first important step is the Login point, which verifies the user and directs

them to either the Admin process or the User process. This separation helps protect administrative functions while keeping basic reporting features available to regular users.

If the person is recognized as an Admin, they gain access to Manage Actions. Through this path, they can add new building records, update existing information, or delete outdated records. These tasks are connected to the CitySafe_DB, which serves as the system's main database. Admins can also query and view reports to monitor system activities and building data.

If the person follows the User path, the process is simpler and mainly centered on retrieving information. Users can submit a Query Report Data request to access details stored in the CitySafe_DB and display them as a report. In either case, the workflow ends with Admin Logout or User Logout, followed by Session End, which closes the system and keeps the data secure.

Individual Contributions

Aguirre — Data Lead

Architected the Persistence Layer, focusing on SQL Server database design and ADO.NET connectivity. Ensured backend synchronization by maintaining the GitHub repository and aligning SQL parameters with the system's business logic.

Iloseo — UI Lead

Directed the Presentation Layer, designing the Windows Forms interface and Microsoft Report Viewer integration. Focused on user experience by mapping UI elements to backend services and ensuring data trends were correctly visualized.

Vera — Logic Lead

Developed the Application Logic and Security, implementing OOP modules, business rules, and role-based access control. Managed system robustness through strict data validation and the integration of logic modules into the UI.

GITHUB REPOSITORY LINK: https://github.com/424000756-design/20MnsLate_SDG11_Project.git